



OPERATING SYSTEM

Memory Management



Concepts covered

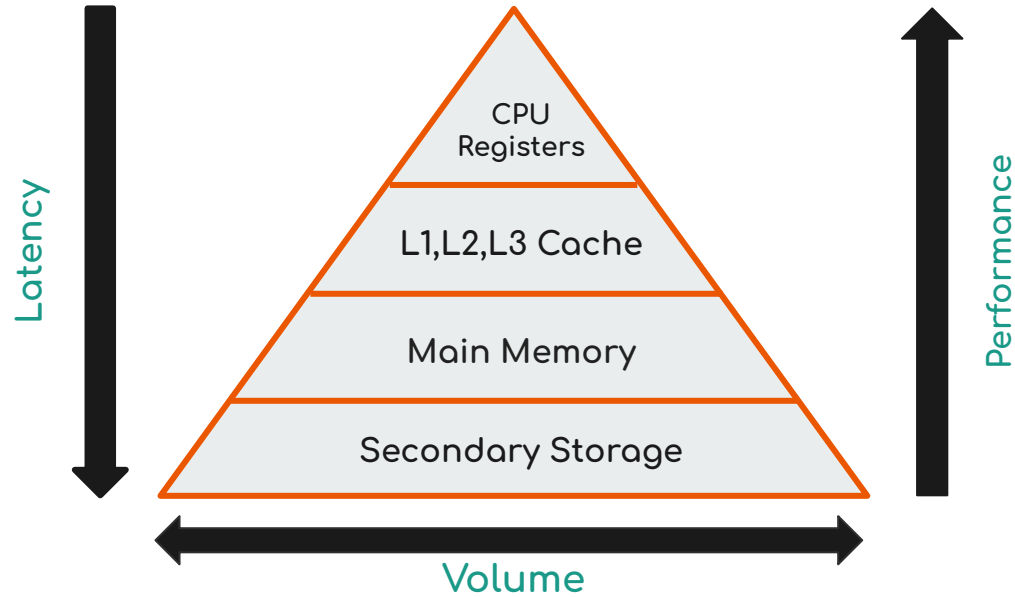
- Background
- Swapping
- Contiguous memory allocation
- Segmentation
- Paging
- Structure of page table
- Example architectures

Background

- A program must be brought from **disk to main memory** and placed within process address space for it to be run.
- A program can be written using machine language, high level language or assembly language.
- **Main memory** and **CPU registers** are the only storage entities that a CPU can access directly
- The CPU fetches instructions from main memory according to the value of **program counter**.
- Typical **instruction execution cycle** - Fetch instruction from memory, decode instruction, Operand fetch, storage of result in main memory

Background

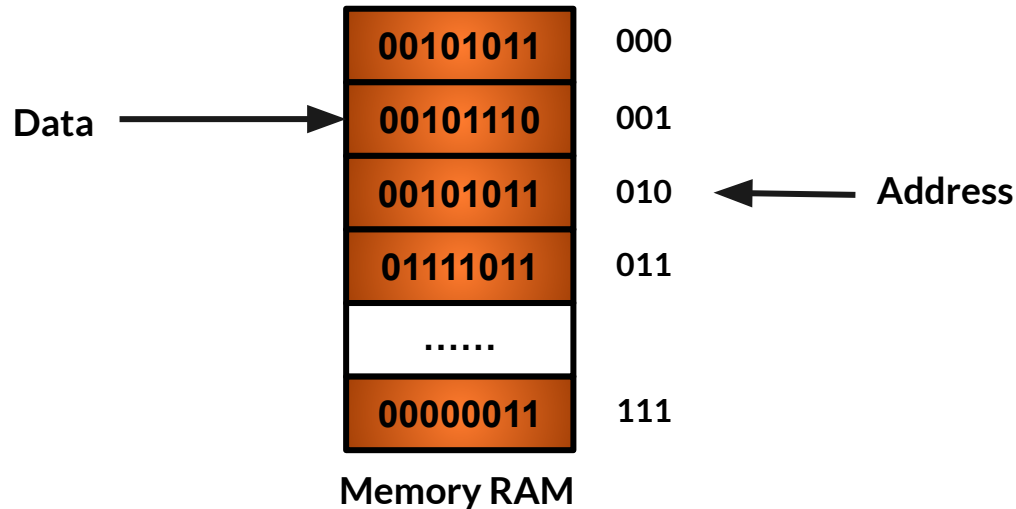
- Memory Hierarchy



Background

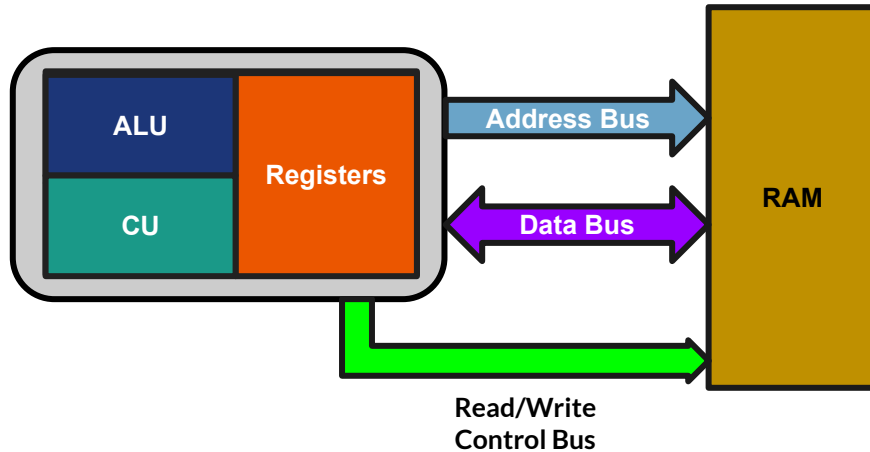
- Memory addresses

In computer memory, the **basic memory unit is a byte** (8 bits) and each byte will have **unique memory address**.



Background

Processor Architecture - Birds Eye View

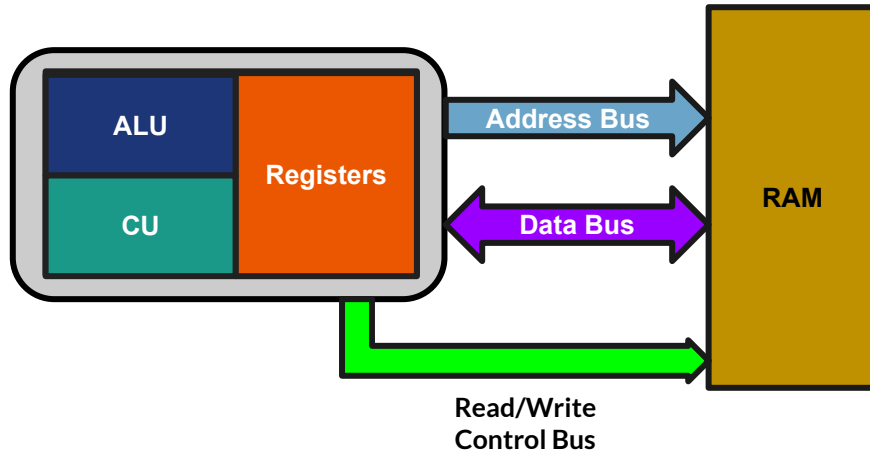


ADDRESS BUS

- A computer with a **32-bit address bus** can directly **address 4GB** of physical memory
- one with **36 bits** can address **64GB**
- The address bus is **uni-directional**.
- It is concerned with passing an address one way, from the CPU to RAM.
- The sole purpose of an address bus is to **identify the address of the location in cache or main memory** that is to be read from or written to.

Background

Processor Architecture - Birds Eye View

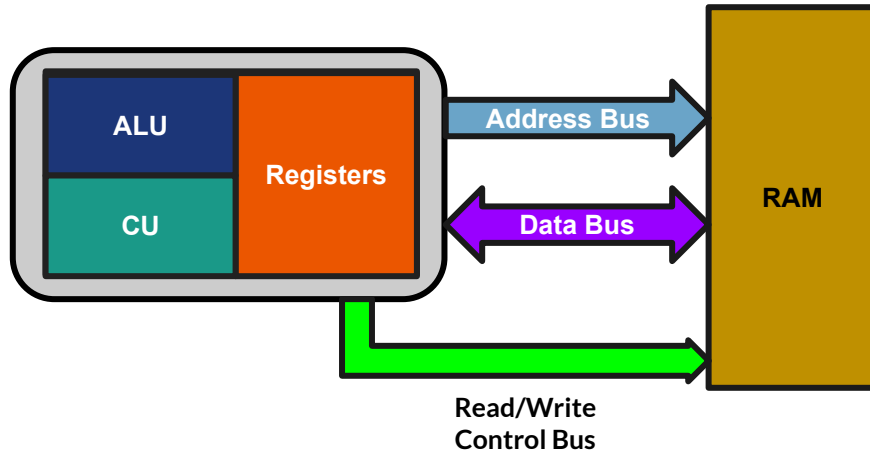


DATA BUS

- The data bus is a wired connection dedicated for the **transmitting the data** between **CPU** and other **H/W components**
- The common data bus include 8-bit, 16-bit, 32-bit or 64-bit.
- Width of the bus determines the **number of bits the bus can carry** at a time. For example, a 32-bit data bus can carry 32-bits of information at a time

Background

Processor Architecture - Birds Eye View



CONTROL BUS

- The control bus is used to **transfer control signals from one component to another**
- The control bus contains a control line for **write** instructions and a control line for **read** instructions.
- When the CPU writes data to the main memory, it **transmits** a signal to the **write command line**.
- The CPU also sends a signal to the **read command line** when it needs to **read**.

Background



Processor Architecture - Birds Eye View

A memory unit only sees the stream of one of the following

- **Address + read requests** (Eg. Load memory location 20010 into register no. 8)
- **Address + data and Write request** (Eg. Store content of register 6 into memory location 34002)

Memory unit does not know how these addresses were generated

Registers can be accessed in one CPU cycle (or less)

Completing memory access may take many cycles of CPU clock. In such case processor needs to stall since it does not have required data to execute instruction.

Background

Processor Architecture - Birds Eye View

- Cache memory sits between main memory and CPU registers to deal with the stall issue

Every time bring water from lake in sprinkler
- Time Consuming



Long distance



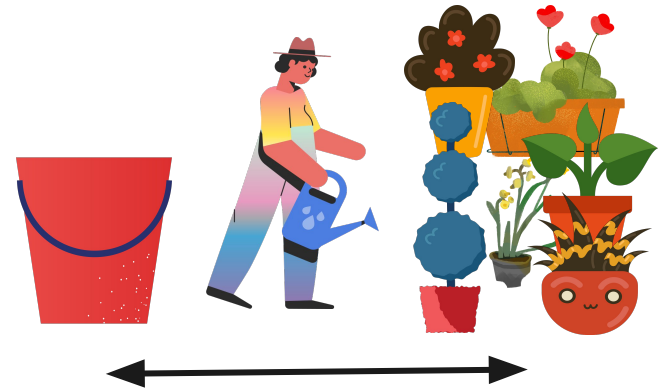
Background

Processor Architecture - Birds Eye View

- Cache memory sits between main memory and CPU registers to deal with the stall issue



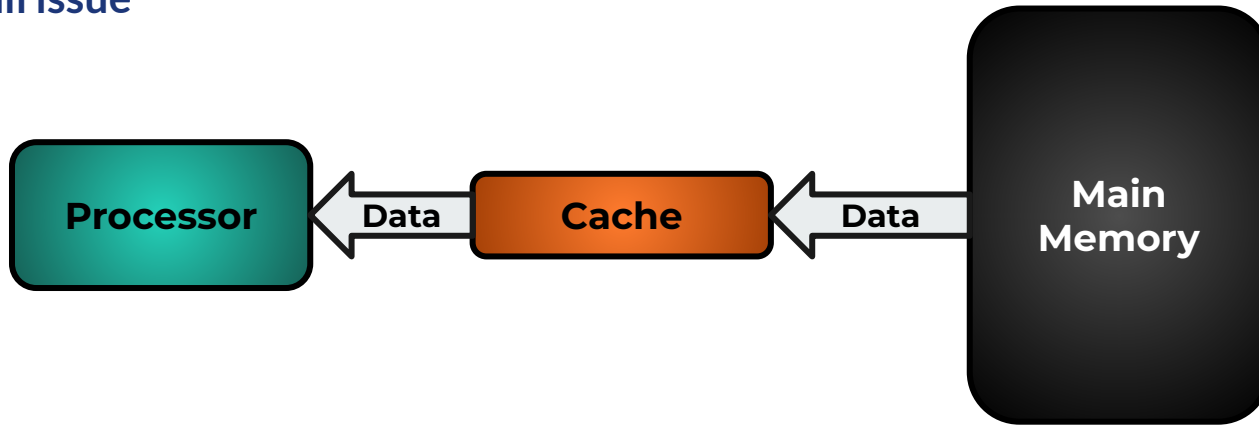
Bring water in a bucket once and fill sprinkler from bucket water.



Background

Processor Architecture - Birds Eye View

- Cache memory sits **between main memory and CPU registers** to deal with the stall issue



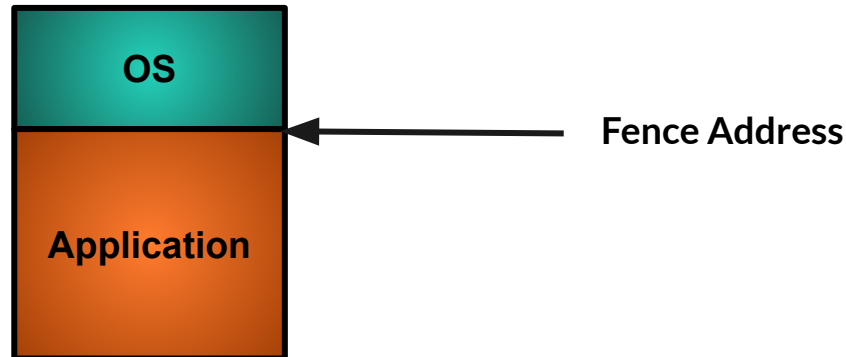
Background

Processor Architecture - Birds Eye View

- **Protection** of memory is required to ensure correct operation.

User Process cannot access the OS memory area

A user process cannot access the memory address space of other process

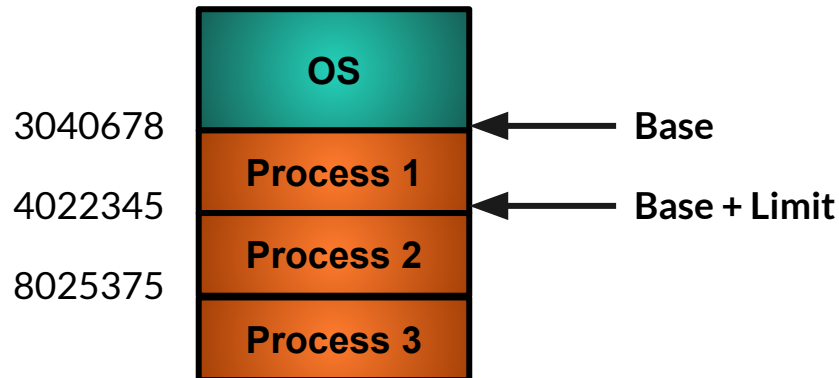


Background

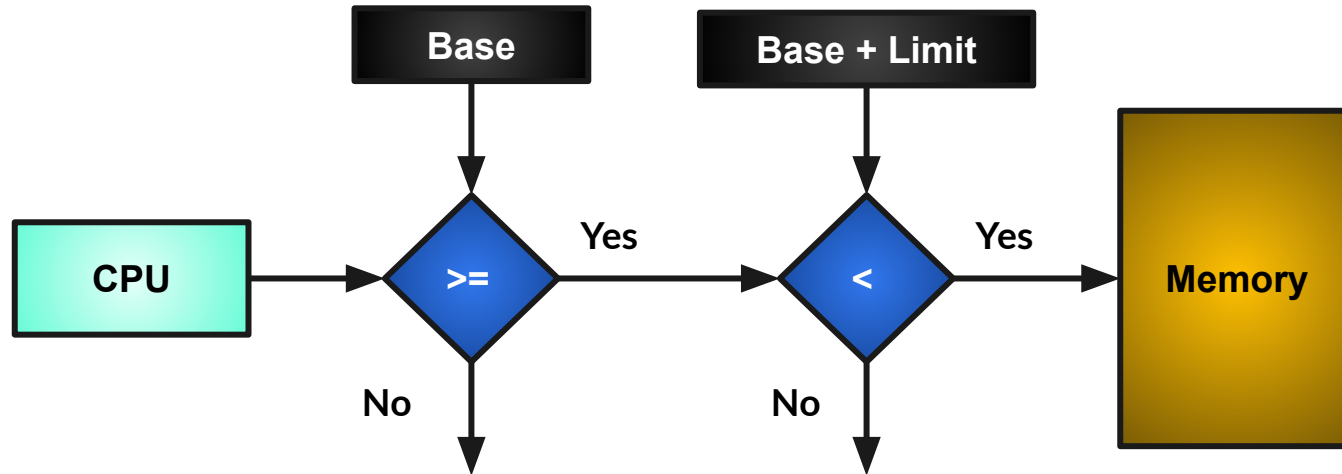
Processor Architecture - Birds Eye View

A **base register** (holding a smallest legal physical address of a program in memory) and **limit register** (specifies the size of the program) define the boundary of a program in the memory.

CPU must check every address generated by the user code is between **base** and **base + limit** for that user



Hardware Access Protection



Addressing Error

Any address generated by CPU goes through two checks, first whether generated address is valid or not and second address is within limits or not

Address Binding in Operating System



RAM



HDD

- Address binding relates to how the code of a program is stored in memory.
- Whenever a program needs to be executed, the code and data related to that program needs to be stored into main memory
- During this transfer from HDD to main memory program resides in a job queue.
- When we write a program it will be stored in HDD and CPU knows exact address of the program on HDD
- During execution of the program it may get loaded into any location in the RAM
- To map the addresses of HDD to main memory addresses is called as address binding

Address Binding in Operating System

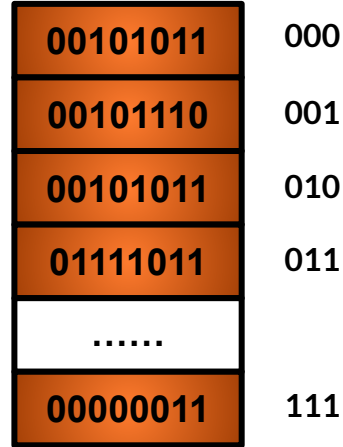


```
var a;  
var b;  
.  
.  
.  
goto: l4  
.  
.  
l4: calc(X);  
.  
.  
l9: end
```

Types of Addresses - Symbolic Address

- A program (**source code**) is written in a high level language such as java
- Initially it resides into the secondary storage i.e. HDD
- The addresses used in **source code** are referred as **Symbolic addresses**
- **The variable names, constants, instruction labels** are referred as Symbolic Addresses
- The symbolic addresses are for the **programmers point of view**

Address Binding in Operating System



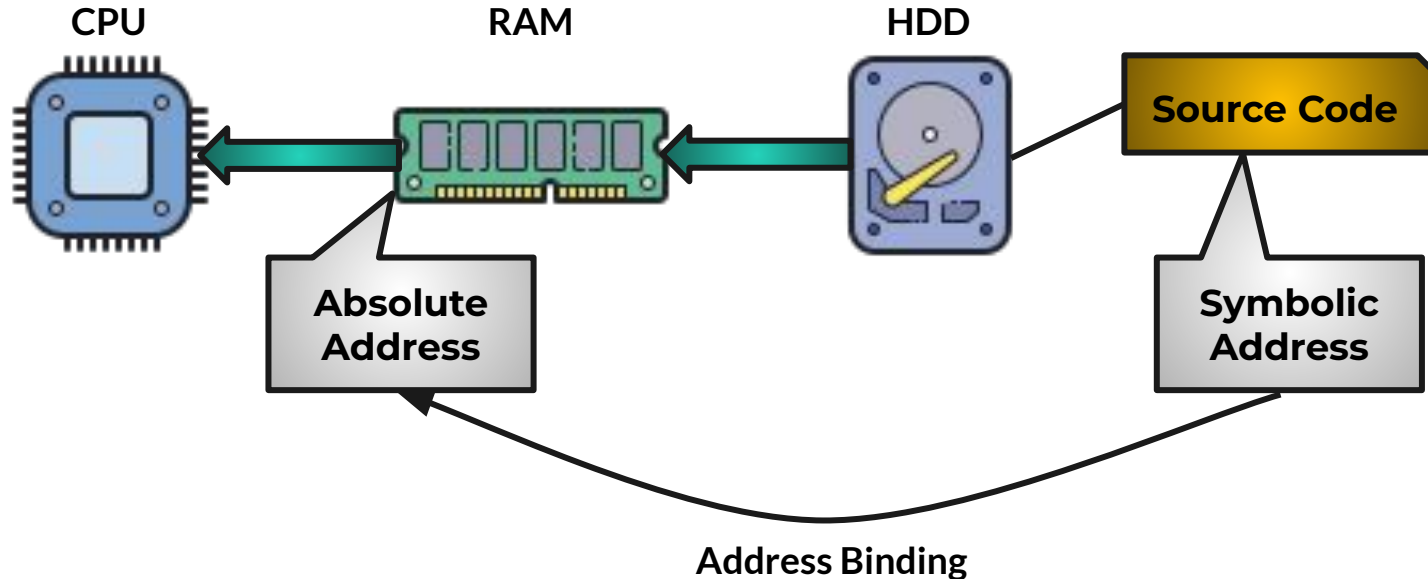
Memory RAM

Types of Addresses - Absolute Address

- Fixed address in the memory (RAM) is called as **absolute address**
- Alternatively known as a **direct address, machine address, or real address**, an absolute address is an exact memory address utilized by hardware devices or software programs.
- In short the address of memory (RAM) is an absolute address

Address Binding in Operating System

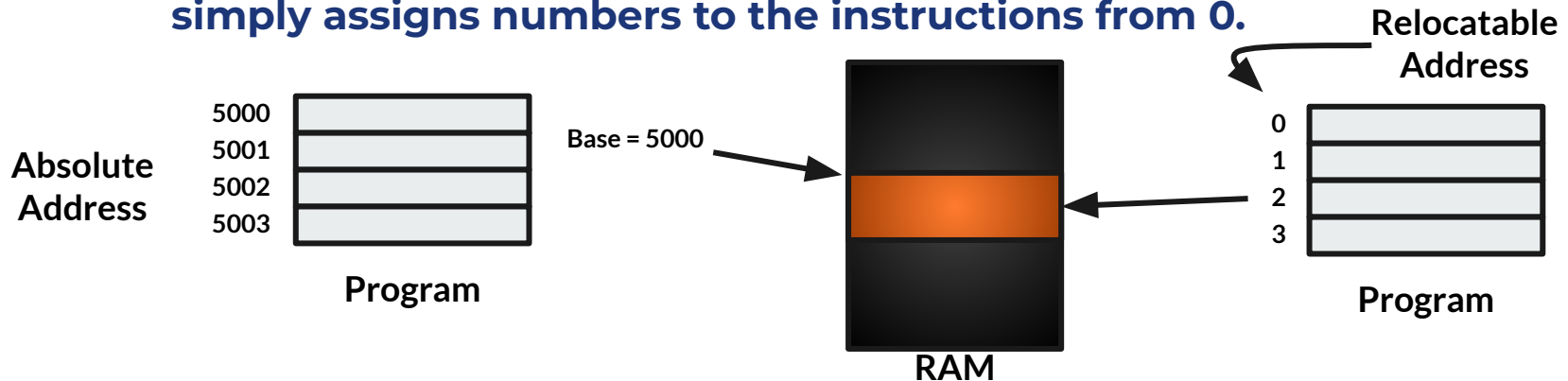
- Mapping of **Symbolic Address to Absolute address** is called as address binding



Address Binding in Operating System

- Types of Address Binding

- 1. Compile Time-** If it is known at compile time where process or program will reside in main memory then absolute address can be generated. In this case symbolic address is bind to **absolute address** at compile time
- 2. Load Time-** If it is not known at compile time, where program gets loaded into memory then **relocatable address** will be generated. It simply assigns numbers to the instructions from 0.

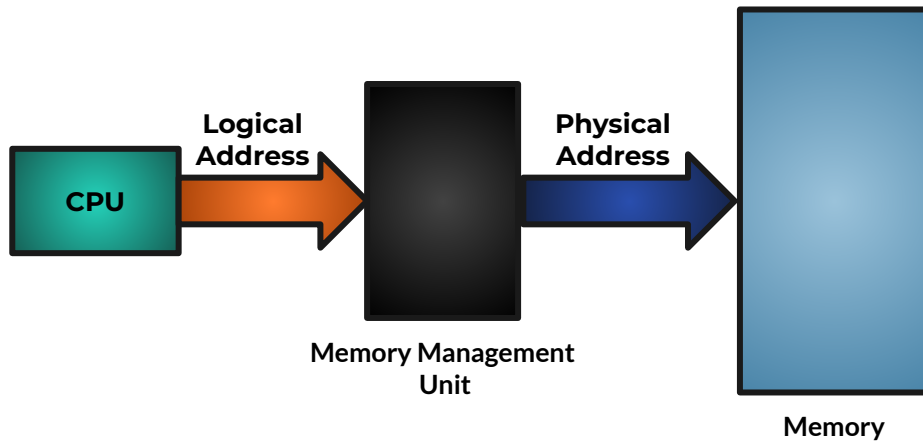


Address Binding in Operating System

- **Types of Address Binding**

- 3. **Execution Time-** The address binding will be postponed even after loading the program into memory. The program will keep on changing the locations in the memory till the time of program execution. This type of Address binding will be done by the processor at the time of program execution.

Logical Address vs Physical Address

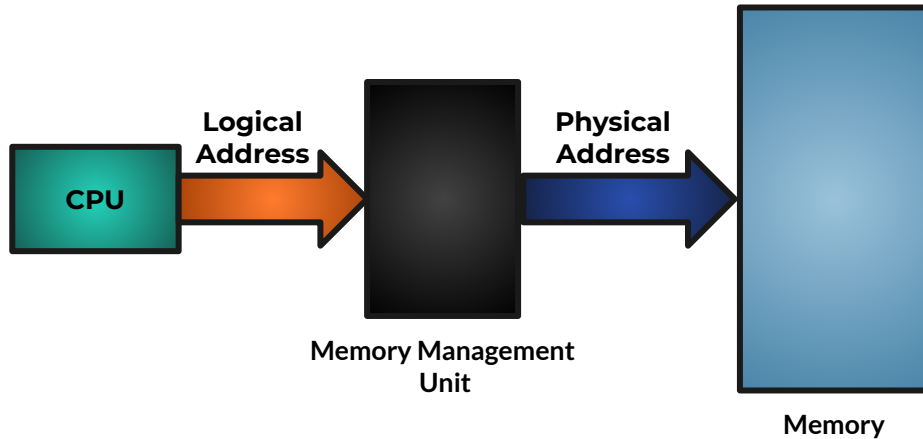


- **Logical Address** is generated by CPU while a program is running.
- The logical address is virtual address as it **does not exist physically**
- This address is used as a **reference to access the physical memory** location by CPU.
- The term Logical Address Space is used for the set of all logical addresses generated by a **program's perspective**.

The hardware device called **Memory-Management Unit** is used for mapping logical address to its corresponding physical address.

* As of now consider MMU as black box, we will discuss it in details in upcoming slides

Logical Address vs Physical Address



- **Physical Address** identifies a physical location of required data in a memory.
- The user **never directly deals with the physical address** but can access by its corresponding logical address.
- The user program generates the logical address and thinks that the program is running in this logical address but the **program needs physical memory for its execution**

The hardware device called **Memory-Management Unit** is used for mapping logical address to its corresponding physical address.

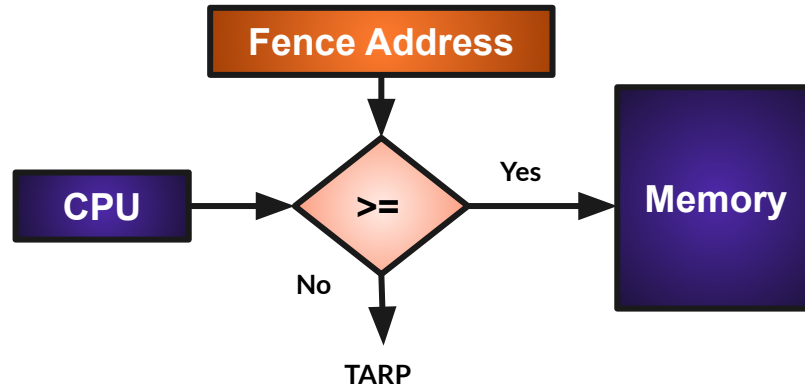
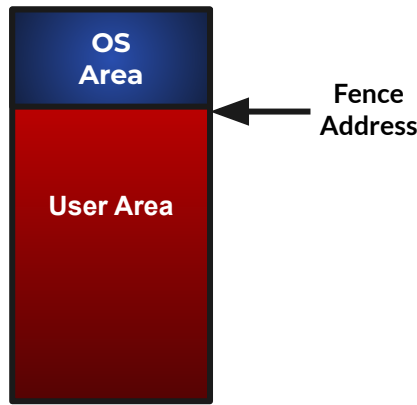
* As of now consider MMU as black box, we will discuss it in details in upcoming slides

Logical Address vs Physical Address

Logical Address	Physical Address
Generated by CPU	Computed by Main Memory Unit
User can view the logical address of a program.	User can never view physical address of program.
The user can use the logical address to access the physical address.	The user can indirectly access physical address but not directly.
Logical address can be change.	Physical address cannot be changed
It is a virtual address	Real memory address
Logical Address Space is a set of all logical addresses generated by CPU	Physical Address Space is a set of all physical addresses corresponding to Logical Address Space

Memory Models

Strategy 1

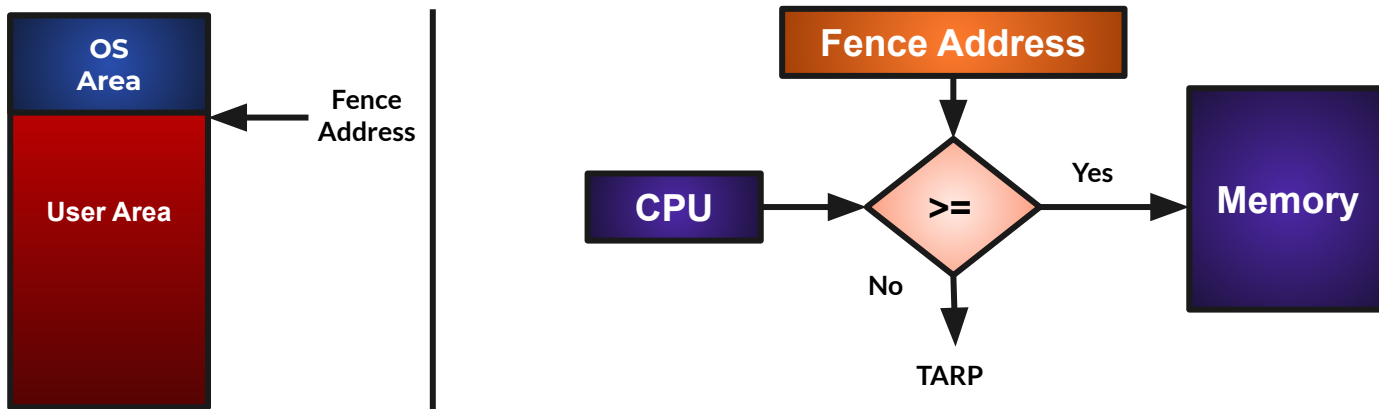


- There are two sections in RAM - **OS area** and **User area**
- Some part of RAM is used to **store operating system** is referred as **OS area**
- Rest of the memory utilised **to execute user processes** is referred as **User Area**.
- A user process **cannot access the memory from OS area**
- The OS area is protected using the **fence address**
- If the address generated by a process is less than fence address then process gets **trapped/terminated**.

Memory Models

Strategy 1

Using Fence Address embedded in Hardware

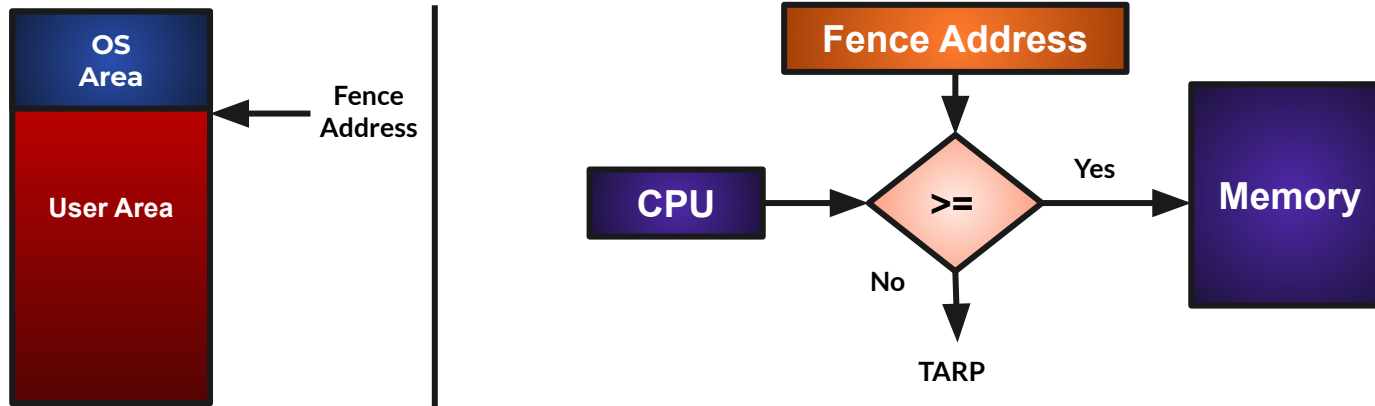


- Fence address can be **stored in hardware** and a comparison operator also be implemented in hardware to speedup the process
- But once fence register is embedded into hardware **it cannot be modified easily**
- In future, due to upgrade the size of OS may get changed. If size gets increased then **OS area may cross fence register** and it will overlap with user area.
- If OS size gets reduced after upgrade then some portion of **memory gets wasted**

Memory Models

Strategy 2

Using **Fence Register** to store Fence Address

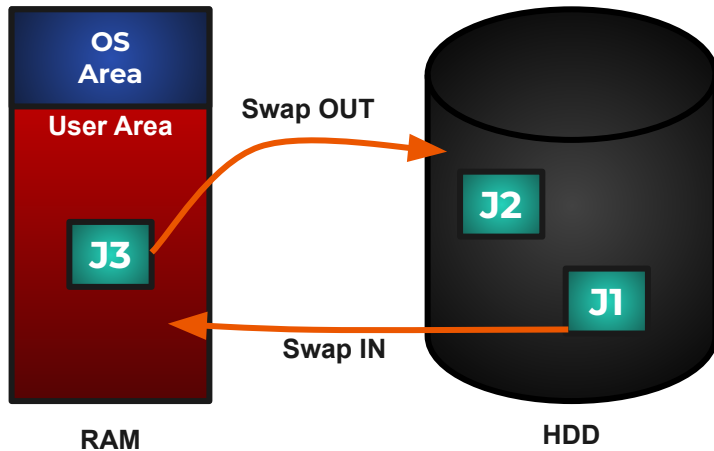


- Another way we can fix a **fence register** to store fence address value
- Whenever OS upgrades the fence register value will get updated accordingly
- However these systems are **not suitable for multi user system** but can be converted into multi user system using swap area

Memory Models

Strategy 3

Multi-user support with **swapping**

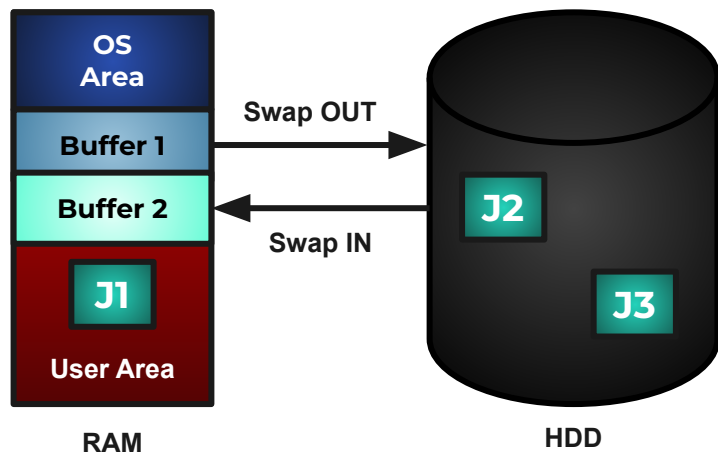


- A memory is divided into two regions OS and User
- If a process gets suspended or terminated a new process from disk will be brought into RAM (**SWAP IN**)
- A suspended process will be stored in the HDD (**SWAP OUT**)
- Thus multi-programming can be achieved
- **Any Drawback?**
Swap in and Swap out operations are very costly

Memory Models

Strategy 4

Multi-user support with **swapping with buffers**



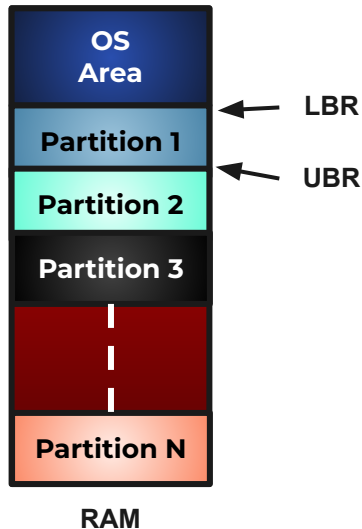
- A memory is divided into four regions OS, Buffer 1, Buffer 2 and User area.
- Process gets executed in user area.
- If a process is **suspended or terminated** then it will be placed in **Buffer 2**.
- New process to be executed will be fetched from **HDD to Buffer 1**.
- Process for execution will be taken into **user area from Buffer 1**.
- **Swapping happens in the RAM** so less overheads
- An efficient memory management unit is required that allows **parallel data transfer** between HDD and RAM

- **Any Drawback?**
Less memory available for process execution
What if a process cannot fit into buffer space?

Memory Models

Strategy 5

Multiprogramming with Fixed number of Tasks (MFT)

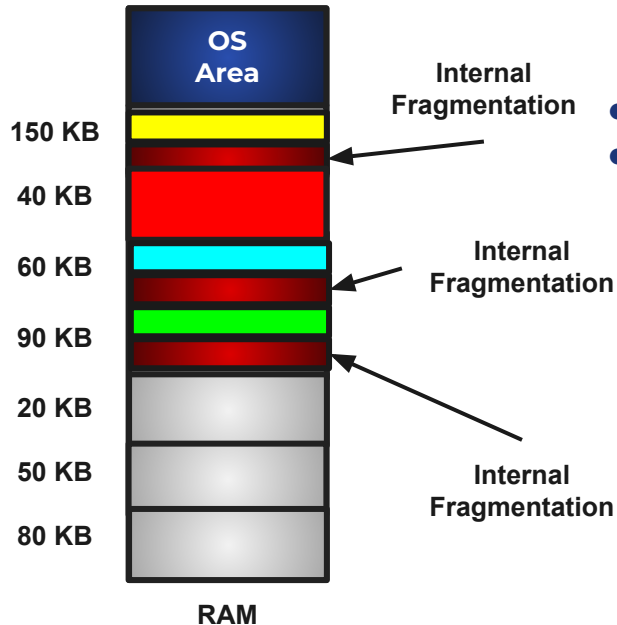


- The User area is divided into number of **partitions of fixed size**
- **Degree of Multiprogramming** depends on **# partitions**
- Switching from one job to another is simply changing the instruction pointer from one partition to another
- **Lower and Upper bound** of each partition is required to maintain **in registers**
- As partitions are of fixed size they **cannot be changed**
- Partitions can be of different sizes
- Whenever a new process to be executed it will be placed in a partition of appropriate partition using two types of algorithms **either first fit or best fit**.

Memory Models

Strategy 5

Multiprogramming with Fixed number of Tasks (MFT)



First Fit Algorithm

- A partition cannot be allocated to more than one process
- Whichever partition starting from first can hold the process is allocated first.

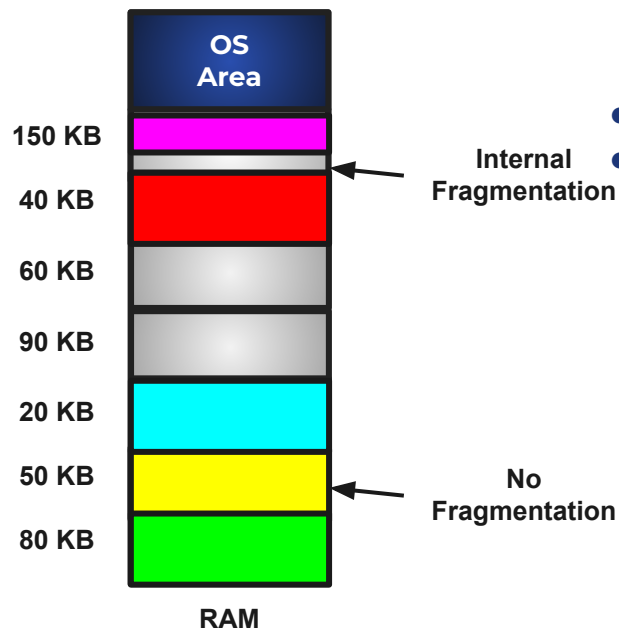


Finally for J5 there is no partition available to place. Though there is 100KB space available in first partition, we cannot place 2 jobs in same partition. **Total 150K Internal Fragmentation**

Memory Models

Strategy 5

Multiprogramming with Fixed number of Tasks (MFT)



Best Fit Algorithm

- A partition cannot be allocated to more than one process
- Search for best fit partition with size greater than or equal to job size and nearest to the job size



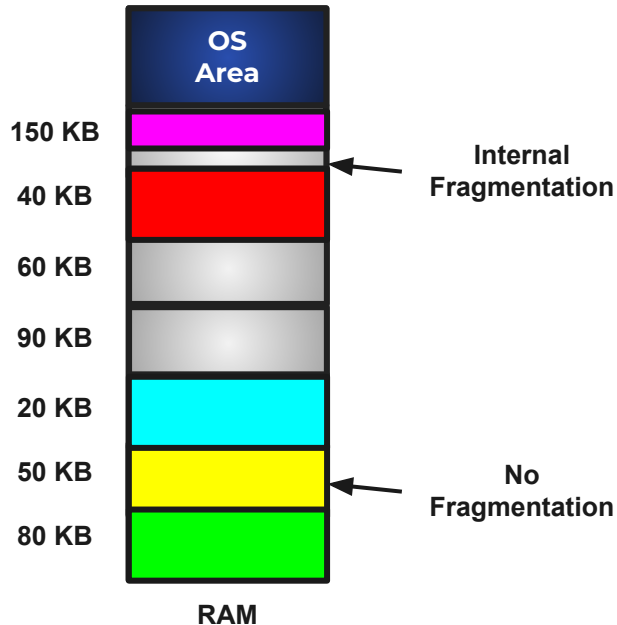
Job Sequence

Total Fragmentation of 50KB

Memory Models

Strategy 5

Multiprogramming with Fixed number of Tasks (MFT)

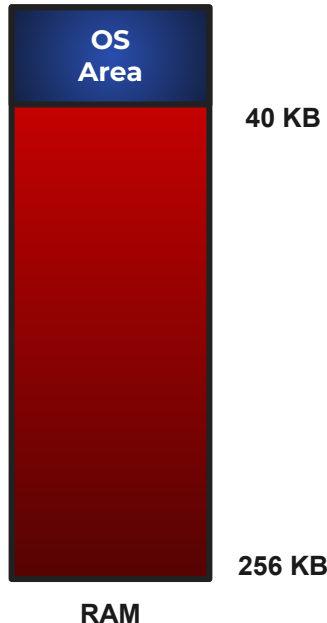


- **Best fit** policy results in **lesser internal fragmentation**, however its complexity is more as it has to find the best suitable partition among all partitions.
- In case of MFT extra information needs to be stored such as number of partitions, size of each partition, process mapping to partition, etc.
- Such type of information is stored in a **partition allocation table**

Memory Models

Strategy 6

Multiprogramming with Variable number of Tasks (MVT)

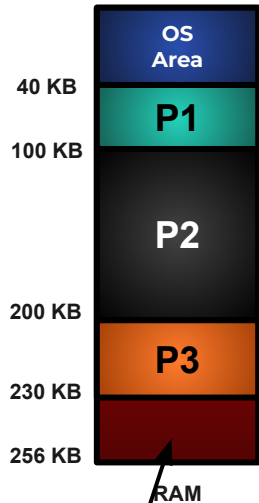


- In case of **MFT**, number of partitions are fixed so **maximum number of task can fit into memory also fixed**
- In case of MVT initially there are only **two partitions one for OS and other for User processes**
- Memory gets allocated to processes based on free space
- Let's assume a memory of size 256 KB out of which 40KB is allocated to Operating System.
- In the following table processes, their memory requirement and CPU burst is given.
- Let's assume the scheduling policy is **FCFS**

Memory Models

Strategy 6

Multiprogramming with Variable number of Tasks (MVT)



26 KB free space
called as External
Fragmentation

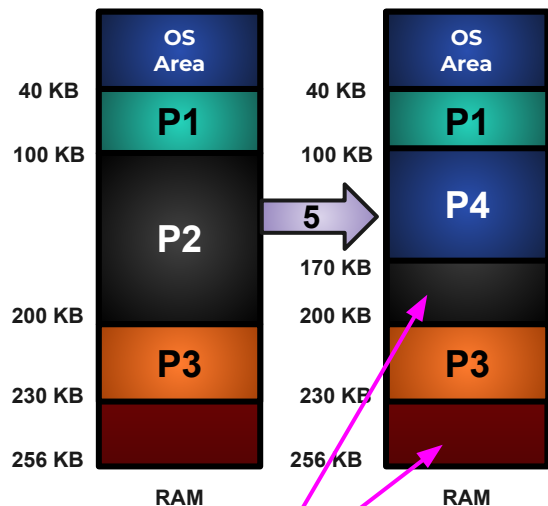
Next process requires 70 KB memory but available is only 26 KB. This memory is unutilized hence it is referred to as external fragmentation

Process	Memory	CPU Burst
P1	60 KB	10
P2	100 KB	5
P3	30 KB	20
P4	70 KB	18
P5	50 KB	15
P6	60 KB	9

Memory Models

Strategy 6

Multiprogramming with Variable number of Tasks (MVT)



P2 completes after 5 units
then P4 can be placed

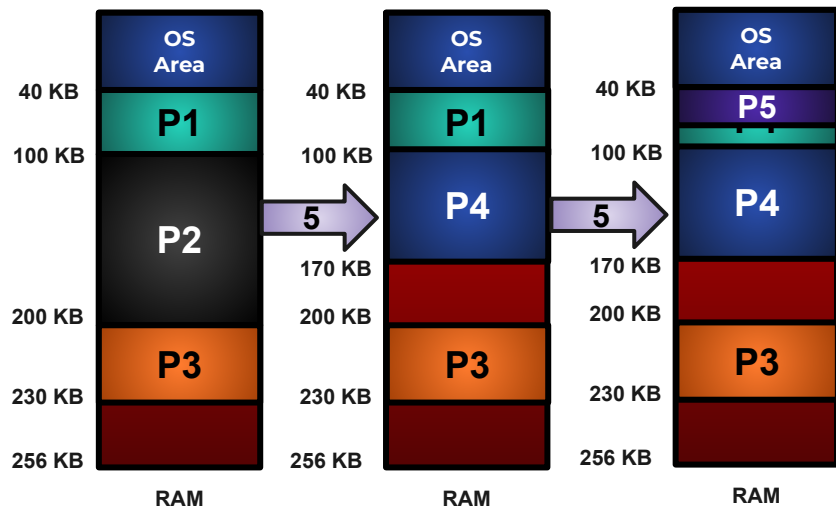
Process	Memory	CPU Burst
P1	60 KB	10
P2	100 KB	5
P3	30 KB	20
P4	70 KB	18
P5	50 KB	15
P6	60 KB	9

Total free space is 56 KB and P5 needs 50KB. Still P5 cannot be loaded as free space is at discrete locations.

Memory Models

Strategy 6

Multiprogramming with Variable number of Tasks (MVT)



P1 completes after 5 units
then P5 can be placed

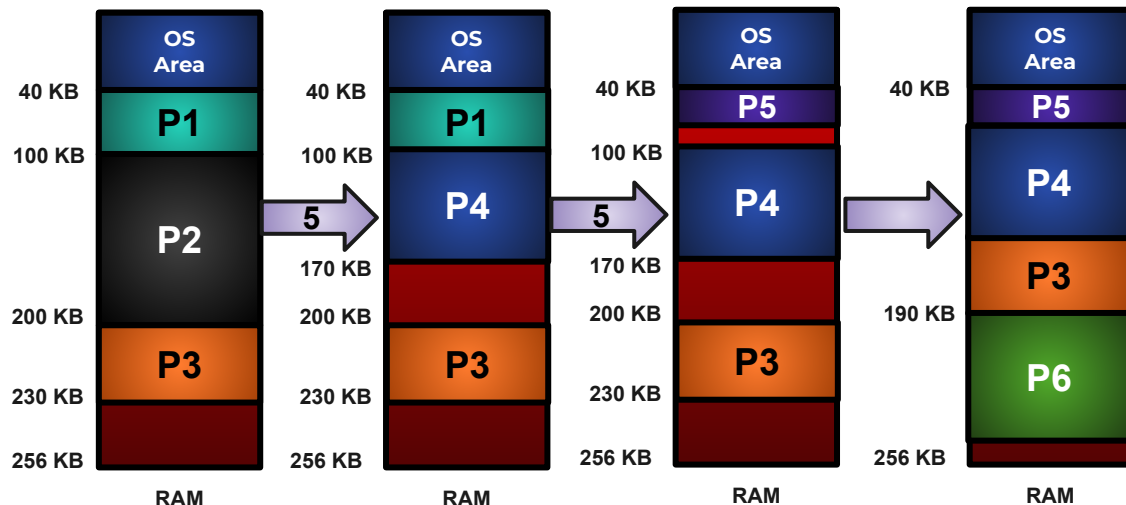
Process	Memory	CPU Burst
P1	60 KB	10
P2	100 KB	5
P3	30 KB	20
P4	70 KB	18
P5	50 KB	15
P6	60 KB	9

- Now P6 needs 60 KB memory which is not available at contiguous location.
- Total free space = 10 KB + 30 KB + 26 KB = 66 KB
- One solution is **Compaction**

Memory Models

Strategy 6

Multiprogramming with Variable number of Tasks (MVT)



Process	Memory	CPU Burst
P1	60 KB	10
P2	100 KB	5
P3	30 KB	20
P4	70 KB	8
P5	50 KB	15
P6	60 KB	9

After Compaction free space of 66 KB is available at contiguous location

Now P6 can be placed in RAM at the end and 6 KB remain unutilized as external fragmentation

Memory Models

Strategy 6

Multiprogramming with Variable number of Tasks (MVT)

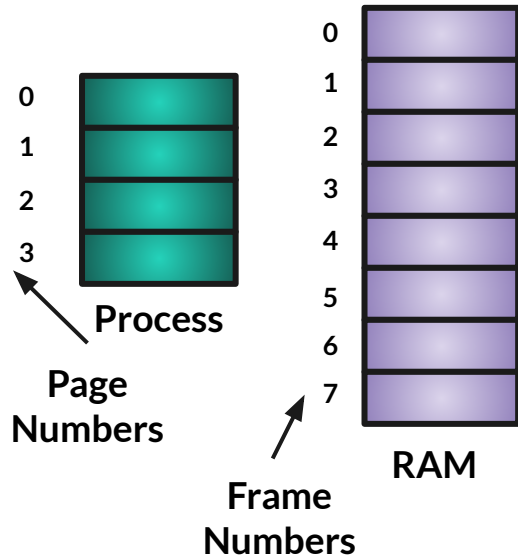
- In case of MVT we need to maintain free space also just like process space
- Free space also has lower bound and upper bound
- **Partition table** keeps track of **upper and lower bound of all process'** address spaces
- **Free area table** keeps track of lower and upper bound of free space.

- **MFT may** results in internal fragmentation due to fixed size partitions
- **MVT** gives better way of memory utilization than **MFT** using **Compaction**
- However compaction is costly because of memory shifting
- During memory shifting CPU operations cannot be performed so waste of CPU cycles
- To overcome the drawbacks of **MFT** and **MVT** another memory management technique can be used that is **Paging**

Memory Models - Non Contiguous Memory Allocation

Strategy 7

Paged Memory Management

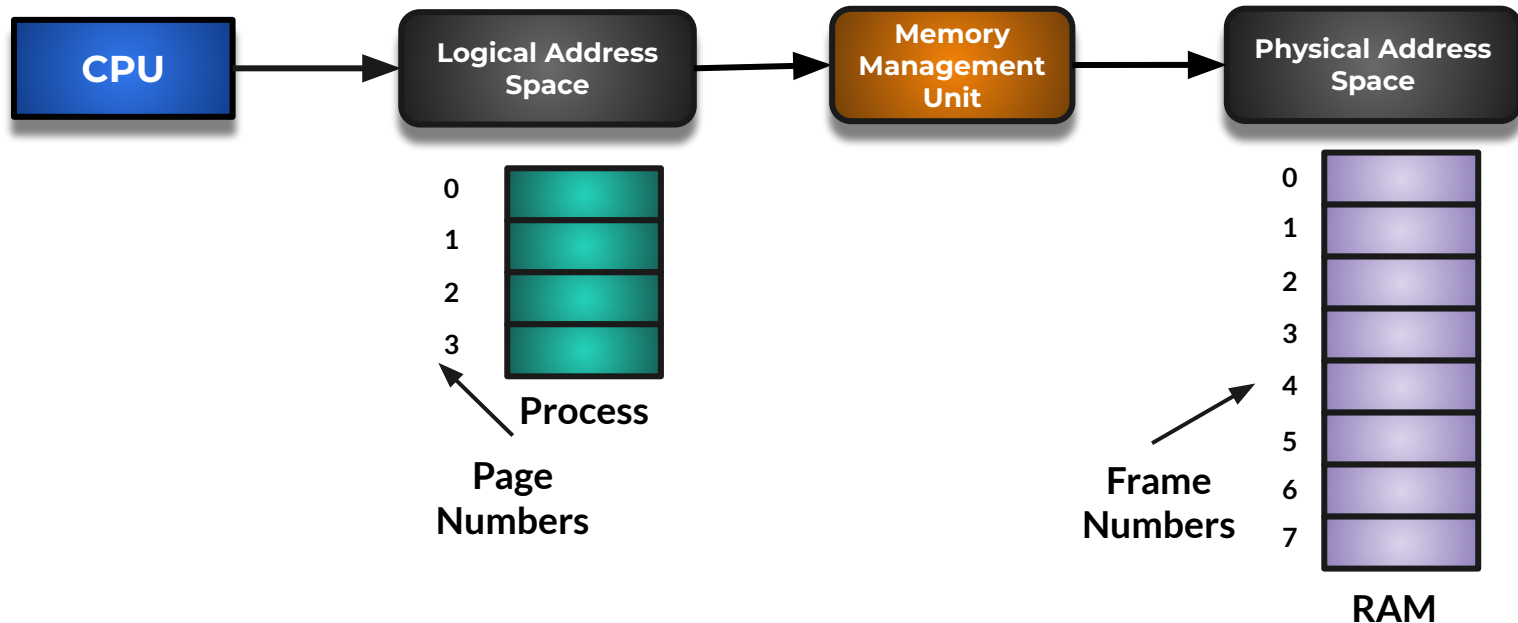


- Every **process** is divided into **pages**, similarly **memory** is divided into partitions of **fixed size equals to page size** called as **frames**.
- All **pages** are of same size except may be last page.
- As **page size and frame size is same**, any page can fit into any frame in memory without fragmentation
- Some of the **frames** are already allocated to **OS** based on the size of OS
- It is allowed to put **any page in any frame**, not necessarily contiguous fashion
- That means **CPU generates logical address in terms of Pages** without caring where actually pages stored in frames (**physical address**).

Memory Models - Non Contiguous Memory Allocation

Strategy 7

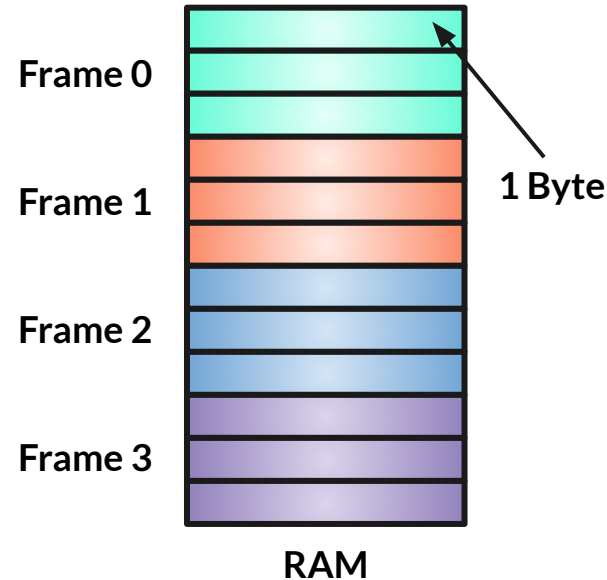
Paged Memory Management Logical to Physical Address Mapping



Memory Models - Non Contiguous Memory Allocation

Strategy 7

Paged Memory Management

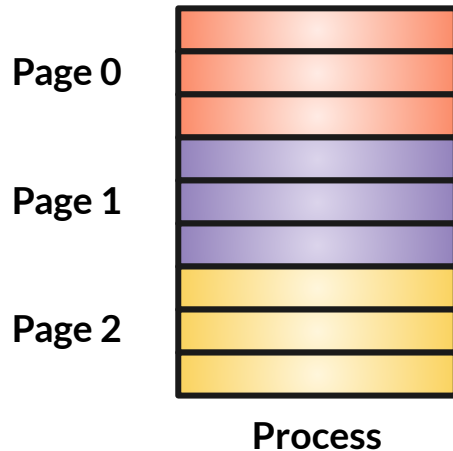


- **Recall** - Memory is divided into **1 byte compartments** and each byte is having its **unique address**.
- A page size or frame size can be of several bytes
- Each frame has a unique number called as frame number
- Each frame shown in figure is of **3 bytes**
- Each process also divided into number of **pages of size 3 bytes**
- Each byte from memory has unique address i.e. **physical address**

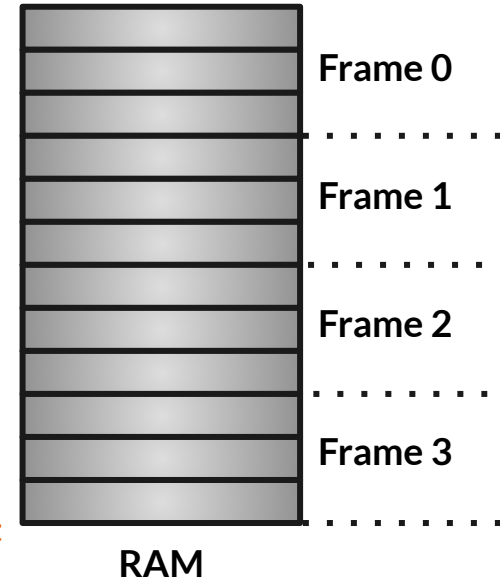
Memory Models - Non Contiguous Memory Allocation

Strategy 7

Paged Memory Management



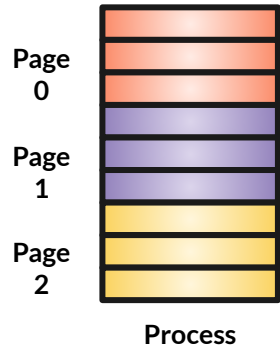
- A process is divided into number of **pages of size 3 bytes**
- CPU knows address in terms of **page numbers and offset within page** this is called as **Logical Address**
- **For example** if CPU generates logical address 7 that means it is from page number 2 and offset within page is 1 i.e. first byte from page 2.
- The pages can be stored in any frame that is free, **no need to store pages contiguously**.
- How to keep track which page of a process stored at which location in main memory?
- We need a **mapping function** that converts **logical address to physical address**
- This mapping is performed by a **memory management unit**



Memory Models - Non Contiguous Memory Allocation

Strategy 7

Paged Memory Management



CPU generates address in terms of page number and offset within page
Let

p = page number
 d = offset within page
 P = page size
 L = Logical Address

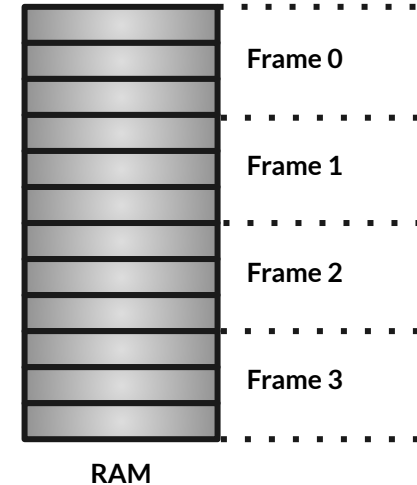
$L = \langle p, d \rangle$

$p = L \text{ div } P$ - This is integer division

$d = L \text{ mod } P$

f = frame number

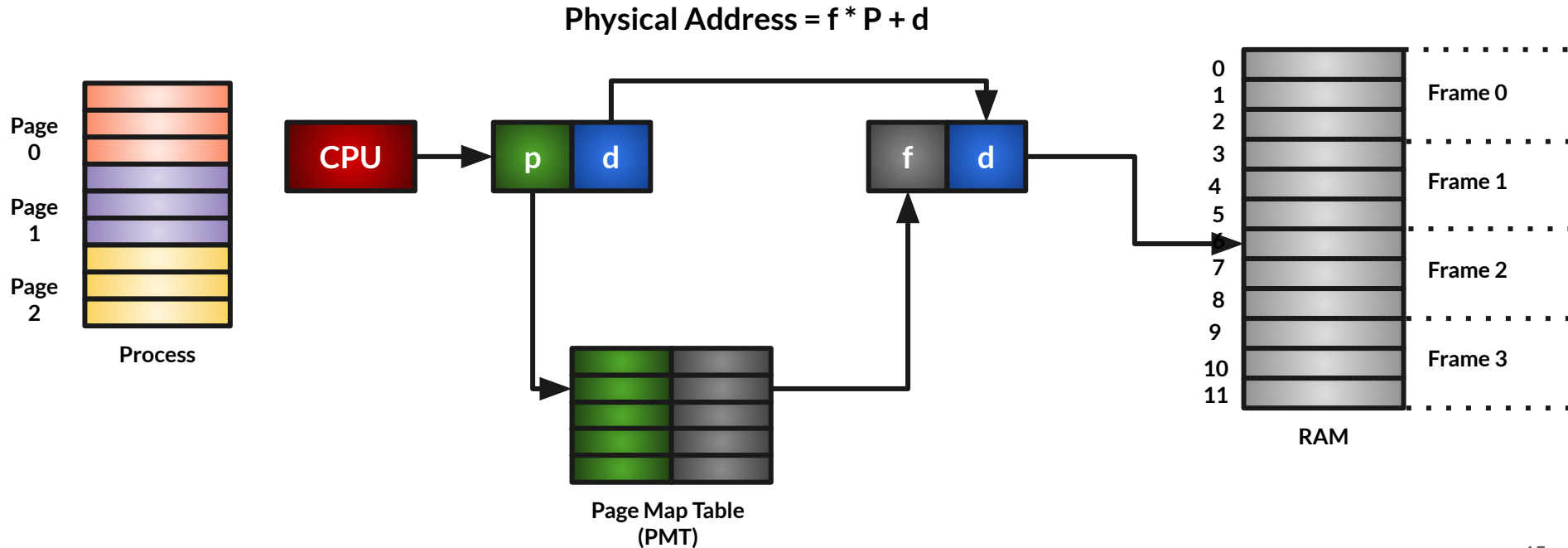
d = offset within frame (As frame size and page size is same so offset for both page and frame is same)



Memory Models - Non Contiguous Memory Allocation

Strategy 7

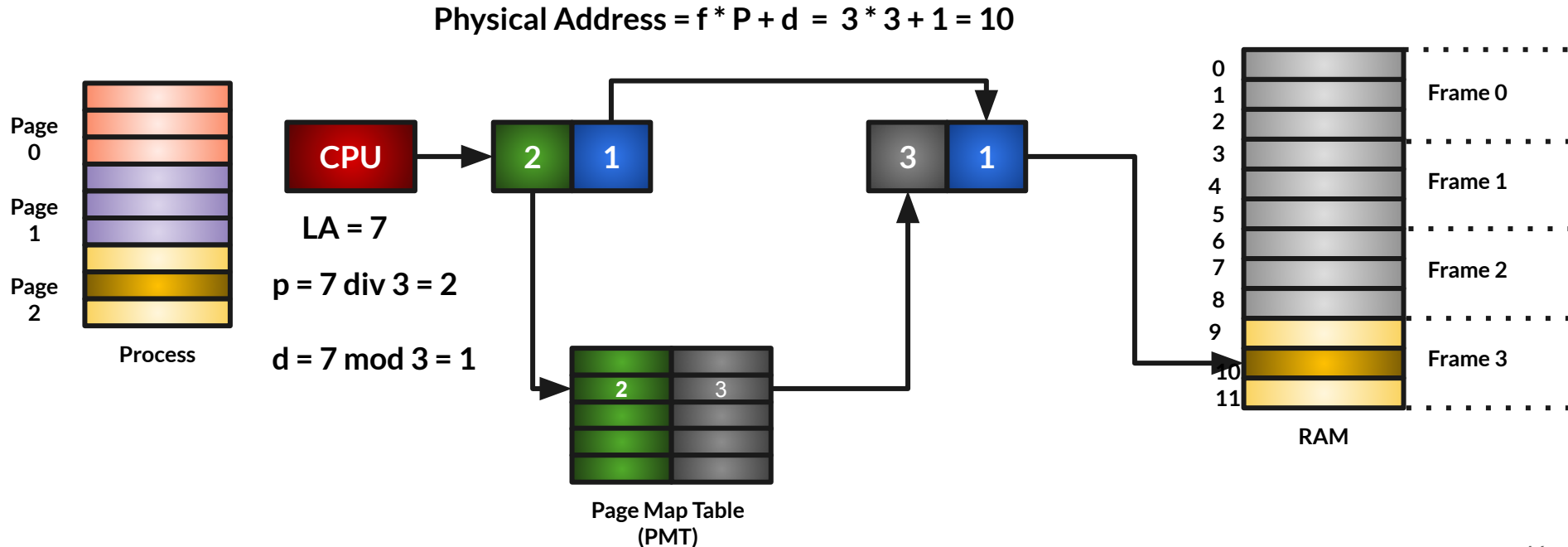
Paged Memory Management



Memory Models - Non Contiguous Memory Allocation

Strategy 7

Paged Memory Management



Memory Models - Non Contiguous Memory Allocation

Strategy 7

Paged Memory Management - Advantages and Disadvantages

- If job is of size 100 KB, we don't need 100 KB free space contiguously
- If page size is 10 KB then we need 10 frames free in memory at any locations
- Using paging technique we can avoid problem of compaction
- Eliminates external fragmentation
- More efficient swapping, just swap out page least likely to be used
- Longer memory access times (page table lookup)
- Internal fragmentation
- No consideration of modular programming

Memory Models - Non Contiguous Memory Allocation

Strategy 7

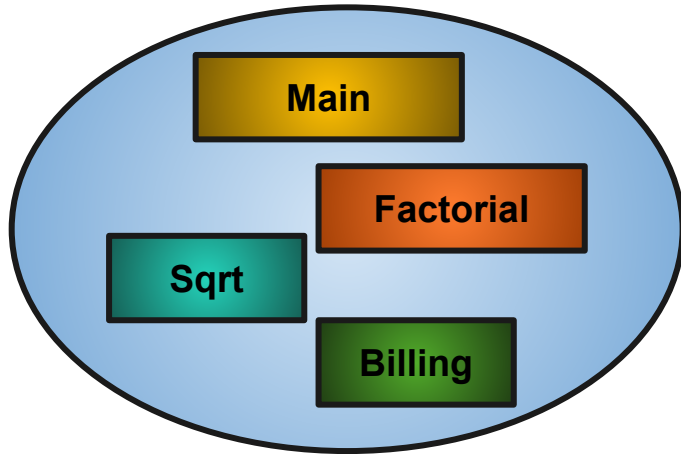
Paged Memory Management

- Software developers prefer **modular programming** because of **reusability** of the code and to provide **semantic structuring** of the code.
- Paging do not consider the modularity of the code. For example it is possible that part of a function is placed in one page and remaining in other page.
- This will lead to severe problem that we will discuss **during virtual memory management**

Memory Models - Non Contiguous Memory Allocation

Strategy 8

Segmented Memory Management

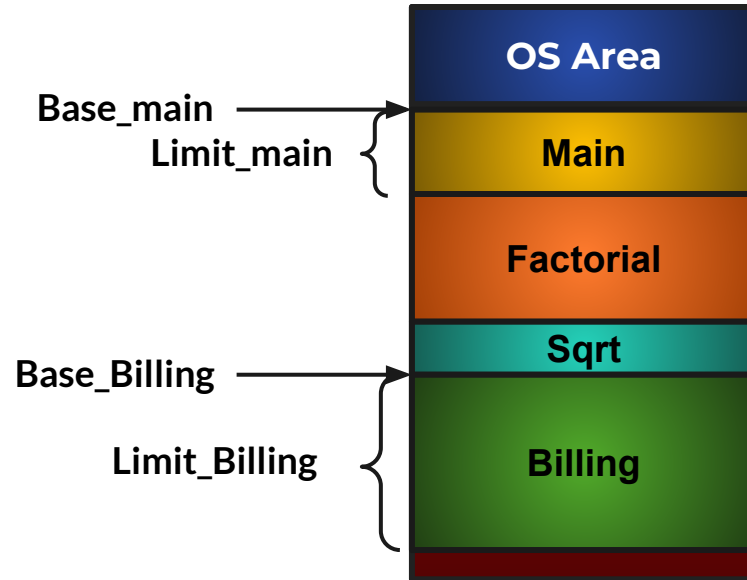
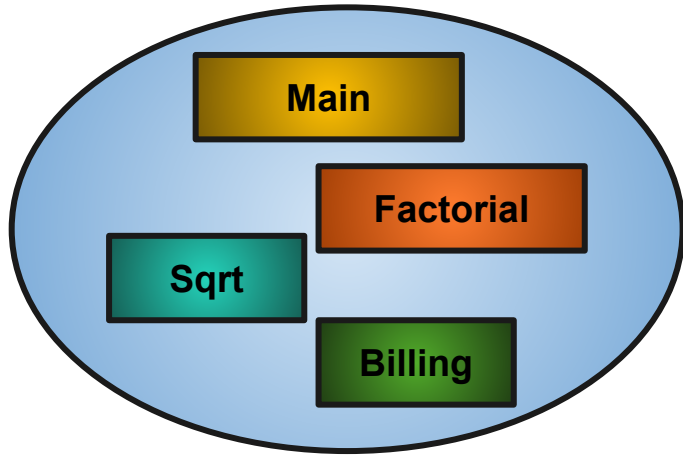


- In a program we can have different functions like `main()`, `factorial()` etc. all comes under **Logical address**
- Instead of breaking a program into pages we can divide based on modules in the program (Segments).
- Each module can be of **different size**
- Basically we want **each module in one block of memory**
- Here memory needs to be divided into **variable partitions**

Memory Models - Non Contiguous Memory Allocation

Strategy 8

Segmented Memory Management



Memory Models - Non Contiguous Memory Allocation

Strategy 8

Segmented Memory Management

- In this case logical address is divided into two parts segment number **s** and offset within the segment as **d**
- Logical address to physical address **mapping is done by memory management unit**
- Memory management unit stores the mapping information in a **segment table**
- **Segment table** keeps track of **base address** and **limit** for each segment
- Generated address should not exceed the range of segment
- If it generates address beyond the limits then it will generate trap and program gets terminated
- You might have observed the error that is **segmentation fault** during programming

Memory Models - Non Contiguous Memory Allocation

Strategy 8

Segmented Memory Management

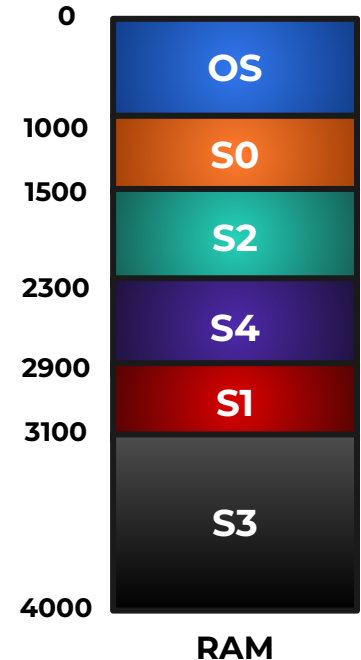
- Each segment can be of **different size**
- **Segment table** keeps track of **base address and limit** of each segment
- **Base address** of segment indicates **starting address** of the segment in the memory
- **Limit** indicates the **size** of that segment
- It is **not necessary** to store segments at **contiguous locations**

Segment Number

0
1
2
3
4

	BA	Limit
0	1000	500
1	2900	200
2	1500	800
3	3100	900
4	2300	600

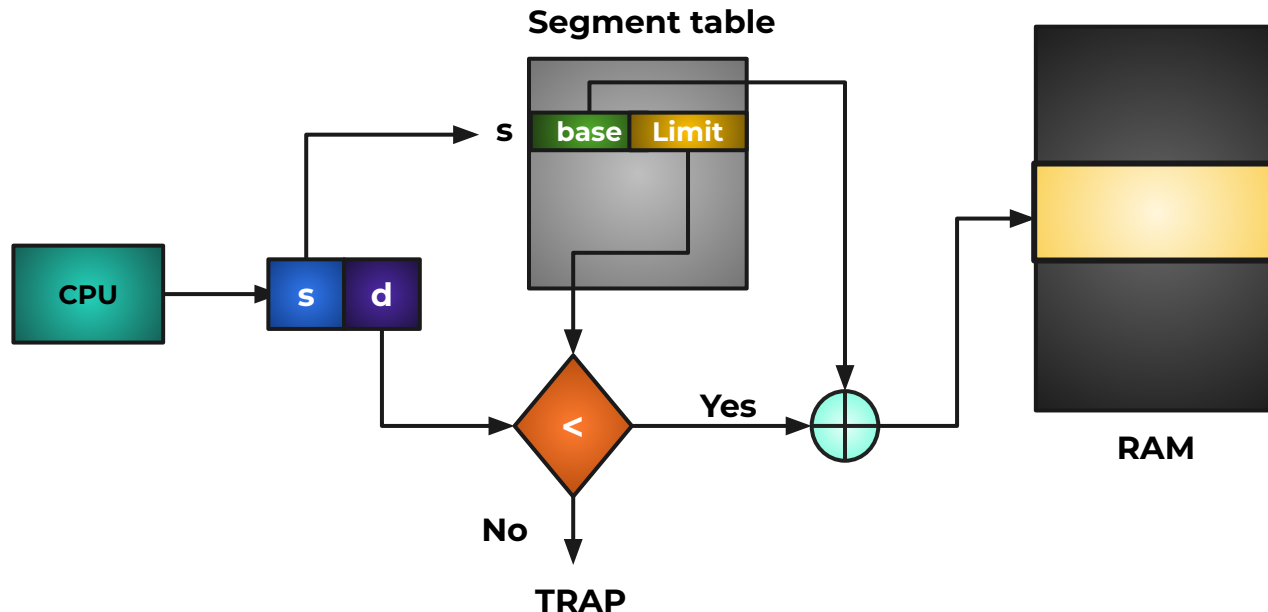
Segment Table



Memory Models - Non Contiguous Memory Allocation

Strategy 8

Segmented Memory Management



- CPU generates **logical address** in terms of **segment number (s)** and **offset**
- Segmentation table has **base** and **limit** value for segment (s)
- Compare offset is less than **limit** or not
- If **No**, then **error** and **terminate** program
- If **yes**, add base address to offset
- Got the **Physical Address**

Memory Models - Non Contiguous Memory Allocation

Strategy 8

Segmented Memory Management - Advantages and Disadvantages

- No internal fragmentation
- Average Segment Size is larger than the actual page size.
- Less overhead
- It is easier to relocate segments than entire address space.
- The segment table is of lesser size as compared to the page table in paging.
- It can have external fragmentation.
- it is difficult to allocate contiguous memory to variable sized partition.
- Costly memory management algorithms.

Memory Models - Non Contiguous Memory Allocation

Strategy 9

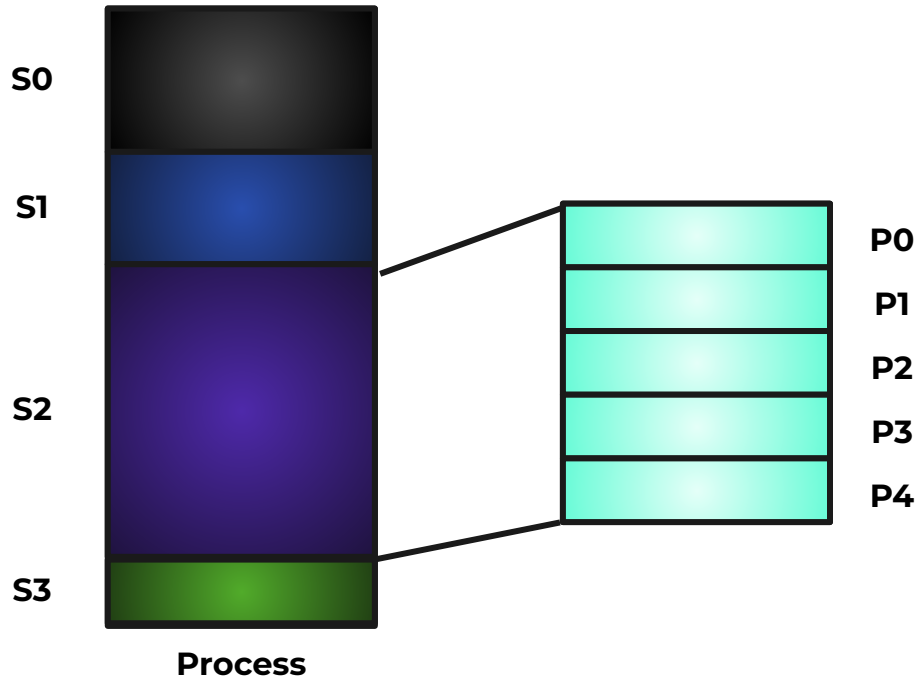
Paged Segmented Memory Management

- This is a combination of **paging** and **segmentation** technique.
- In case of paged memory management technique we have seen for **every user program** there is a **page map table** and if program size is very large then page table size also becomes large.
- In case of **paged segmented memory management** technique we have **page table for every segment**.
- Every **segment** is divided into number of **pages of same size**
- Main memory is divided into number of frames where **page size equals to frame size**.

Memory Models - Non Contiguous Memory Allocation

Strategy 9

Paged Segmented Memory Management

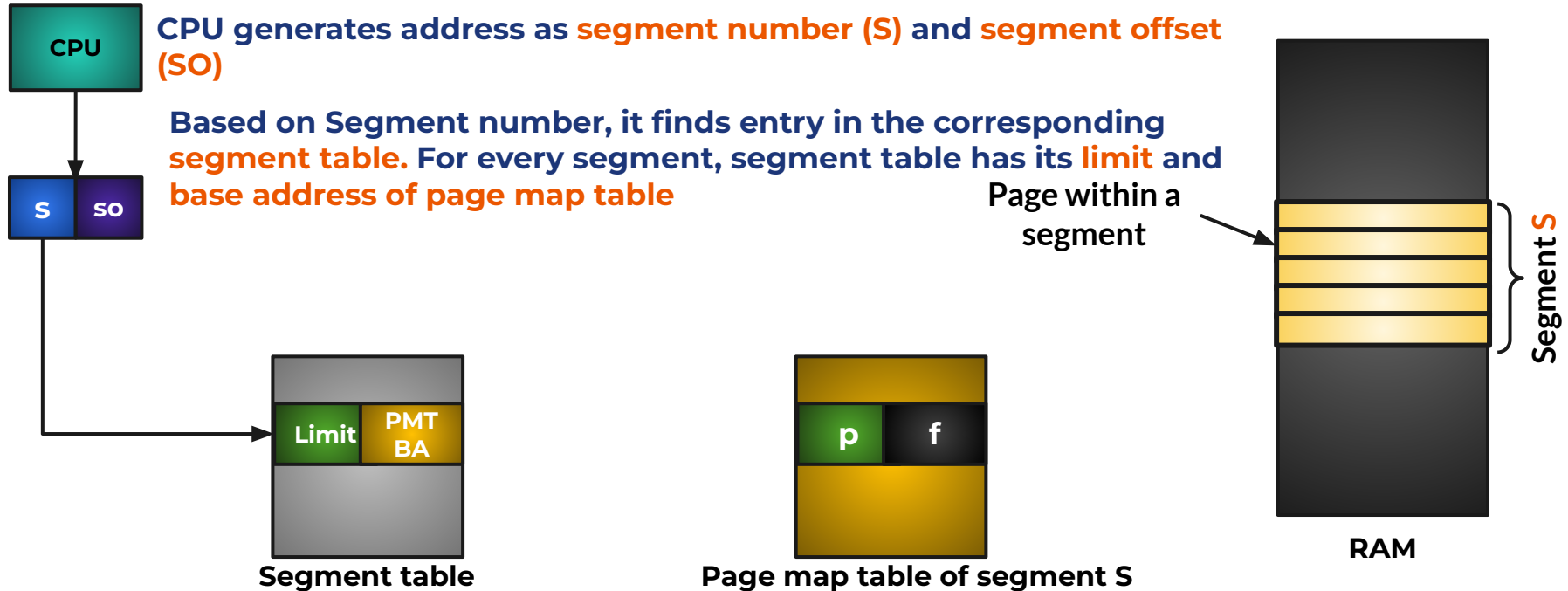


- A process is divided into number of **segments of variable size**
- A segment is further divided into number of **pages of same size**
- In this case we need to maintain 2 types of tables - **One Segment table** and **Page Table** for each segment

Memory Models - Non Contiguous Memory Allocation

Strategy 9

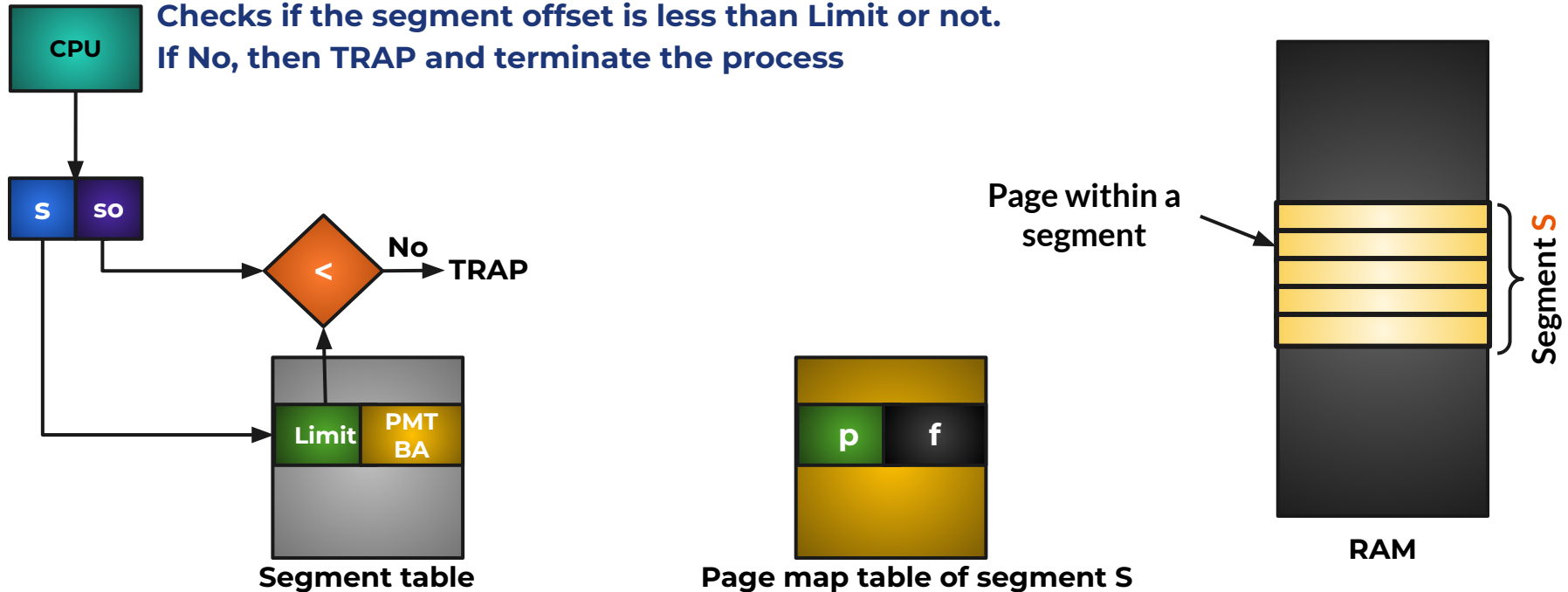
Paged Segmented Memory Management



Memory Models - Non Contiguous Memory Allocation

Strategy 9

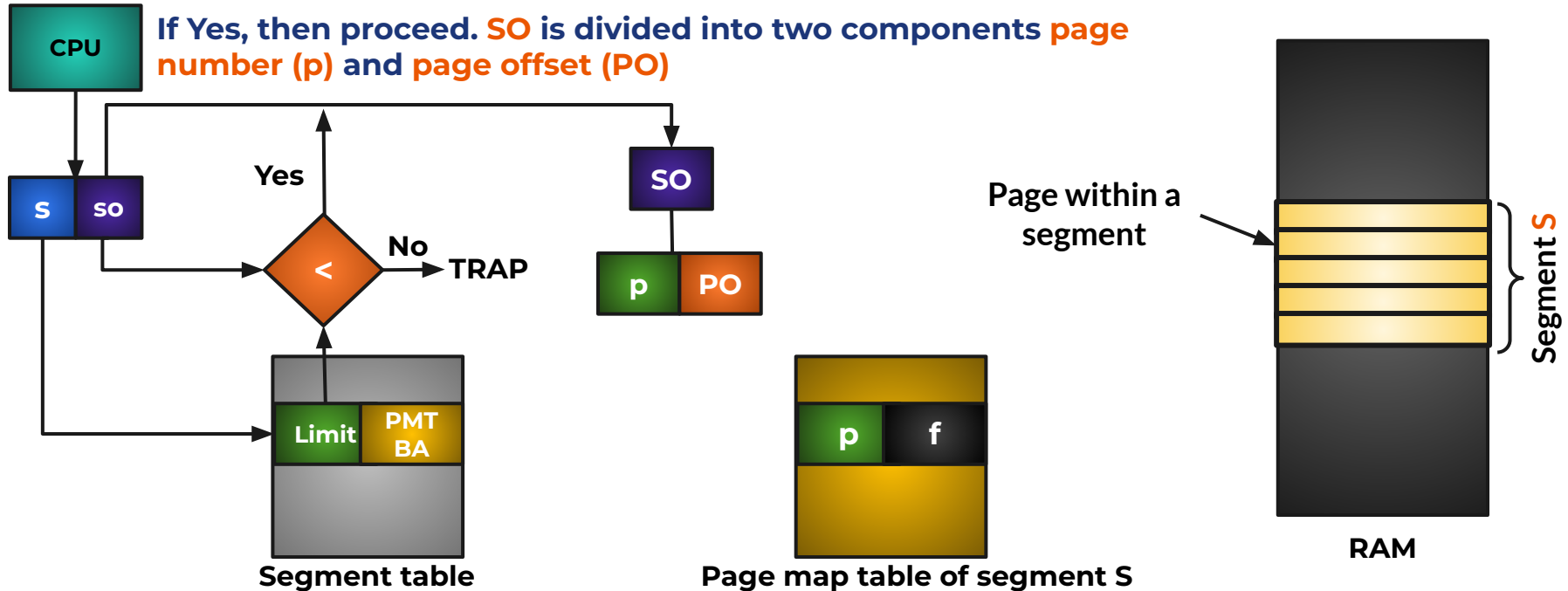
Paged Segmented Memory Management



Memory Models - Non Contiguous Memory Allocation

Strategy 9

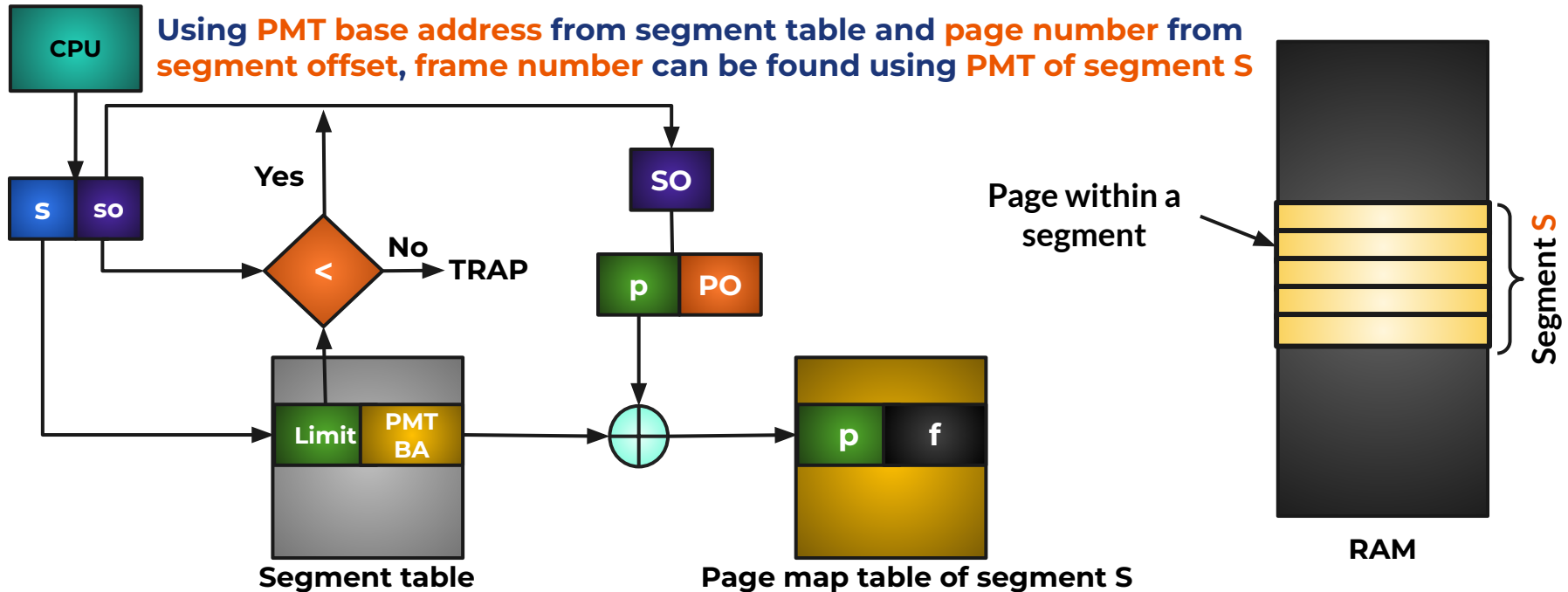
Paged Segmented Memory Management



Memory Models - Non Contiguous Memory Allocation

Strategy 9

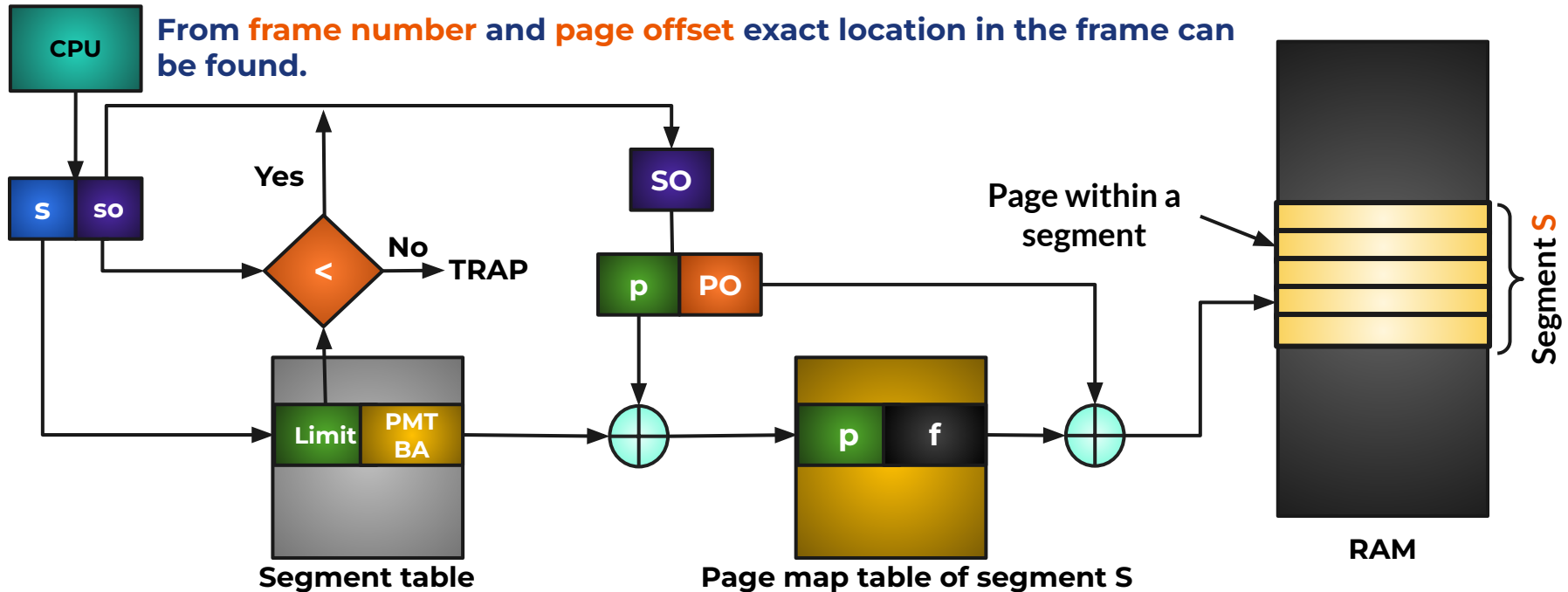
Paged Segmented Memory Management



Memory Models - Non Contiguous Memory Allocation

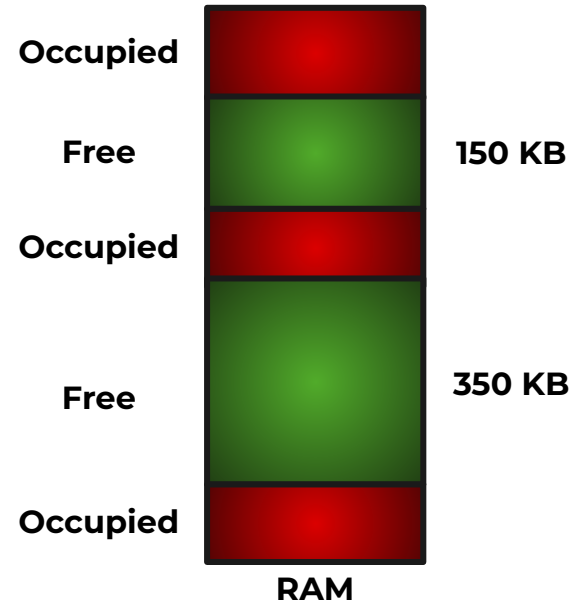
Strategy 9

Paged Segmented Memory Management



Exercise question 1

- Given memory requests from 4 processes are 300KB, 25KB, 125KB, 50KB. There is a variable partitioning method used for memory allocation. Find out which of the following mentioned algorithm allocates memory to all processes. Current status of the RAM mentioned in the diagram.
 1. First Fit
 2. Best Fit
 3. Worst Fit

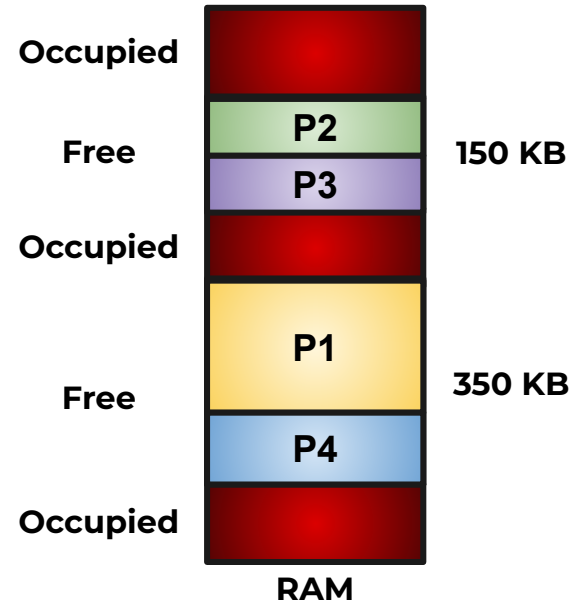
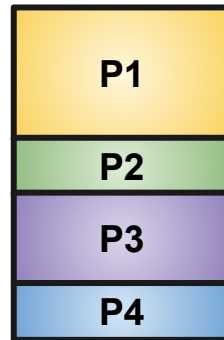


Exercise question 1

- Given memory requests from 4 processes are 300KB, 25KB, 125KB, 50KB. There is a **variable partitioning method** used for memory allocation. Find out which of the following mentioned algorithm allocates memory to all processes. Current status of the RAM mentioned in the diagram. **Processes follow FCFS algorithm.**
 1. First Fit
 2. Best Fit
 3. Worst Fit

First Fit

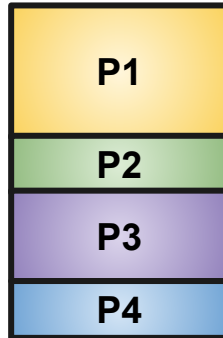
First fit strategy allocates memory all processes



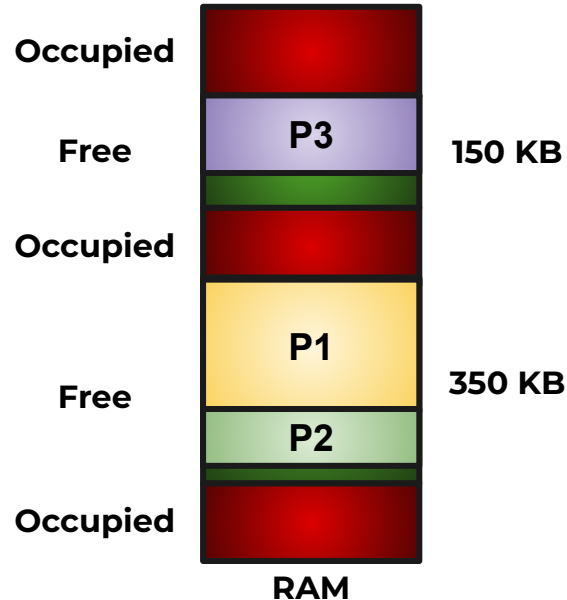
Exercise question 1

- Given memory requests from 4 processes are 300KB, 25KB, 125KB, 50KB. There is a **variable partitioning method** used for memory allocation. Find out which of the following mentioned algorithm allocates memory to all processes. Current status of the RAM mentioned in the diagram. **Processes follow FCFS algorithm.**
 1. First Fit
 2. Best Fit
 3. Worst Fit

Best Fit



**No free
space
for P4**



Formulae



For Main Memory-

- Physical Address Space = Size of main memory
- Size of main memory = Total number of frames x Page size
- Frame size = Page size
- If number of frames in main memory = 2^X , then number of bits in frame number = X bits
- If Page size = 2^X Bytes, then number of bits in page offset = X bits
- If size of main memory = 2^X Bytes, then number of bits in physical address = X bits

Formulae



For Process-

- Virtual Address Space = Size of process
- Number of pages the process is divided = Process size / Page size
- If process size = 2^x bytes, then number of bits in virtual address space = X bits

Formulae



For Page Table-

- Size of page table = Number of entries in page table x Page table entry size
- Number of entries in pages table = Number of pages the process is divided
- Page table entry size = Number of bits in frame number + Number of bits used for optional fields if any

Tricky Question



Q.1 Calculate the size of memory if its address consists of 22 bits and the memory is 2-byte addressable.

Ans:-

We have-

- Number of locations possible with 22 bits = 2^{22} locations
- It is given that the size of one location = 2 bytes

Thus, Size of memory

$$= 2^{22} \times 2 \text{ bytes}$$

$$= 2^{23} \text{ bytes}$$

$$= 8 \text{ MB}$$

Exercise question 2



Consider a machine with **64 MB physical memory** and a **32 bit virtual address space**. If the **page size is 4 KB**, what is the approximate size of the page table?

- A. 16 MB
- B. 8 MB
- C. 2 MB
- D. 24 MB

Exercise question 2



Consider a machine with **64 MB physical memory** and a **32 bit virtual address space**. If the **page size is 4 KB**, what is the approximate size of the page table?

Given-

- Size of main memory = 64 MB
- Number of bits in virtual address space = 32 bits
- Page size = 4 KB

Number of Bits in Physical Address-

Size of main memory

= 64 MB

= 2^{26} B

Thus, Number of bits in physical address = 26 bits

Exercise question 2

Consider a machine with **64 MB physical memory** and a **32 bit virtual address space**. If the **page size is 4 KB**, what is the approximate size of the page table?

Number of Frames in Main Memory-

Number of frames in main memory

= Size of main memory / Frame size

= 64 MB / 4 KB

= 2^{26} B / 2^{12} B

= 2^{14}

Thus, Number of bits in frame number = 14 bits

Number of Bits in Page Offset-

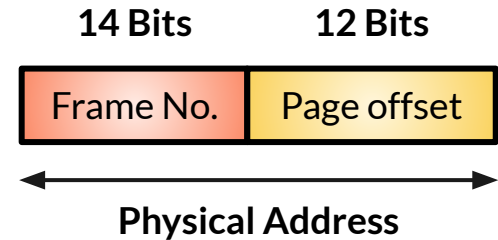
We have,

Page size

= 4 KB

= 2^{12} B

Thus, Number of bits in page offset = 12 bits



Exercise question 2



Consider a machine with **64 MB physical memory** and a **32 bit virtual address space**. If the **page size is 4 KB**, what is the approximate size of the page table?

Process Size-

Number of bits in virtual address space = 32 bits

Thus,

Process size

$$= 2^{32} \text{ B}$$

$$= 4 \text{ GB}$$

Number of Entries in Page Table-

Number of pages the process is divided

$$= \text{Process size} / \text{Page size}$$

$$= 4 \text{ GB} / 4 \text{ KB}$$

$$= 2^{20} \text{ pages}$$

Thus, Number of entries in page table = 2^{20} entries

Exercise question 2



Consider a machine with **64 MB physical memory** and a **32 bit virtual address space**. If the **page size is 4 KB**, what is the approximate size of the page table?

Page Table Size-

Page table size

= Number of entries in page table x Page table entry size

= Number of entries in page table x Number of bits in frame number

= $2^{20} \times 14$ bits

= $2^{20} \times 16$ bits (Approximating 14 bits $\approx$ 16 bits)

= $2^{20} \times 2$ bytes

= 2 MB

Thus, Option (C) is correct.

Practice question

In a virtual memory system, size of **virtual address is 32-bit**, size of **physical address is 30-bit**, **page size is 4 Kbyte** and size of each **page table entry is 32-bit**. The main memory is byte addressable. Which one of the following is the maximum number of bits that can be used for storing protection and other information in each page table entry?

- A. 2
- B. 10
- C. 12
- D. 14

Exercise question 3

A certain computer system has the **segmented paging architecture** for virtual memory. The memory is byte addressable. **Both virtual and physical address spaces contain 2^{16} bytes each.** The virtual address space is divided into **8 non-overlapping equal size segments**. The memory management unit (MMU) has a hardware segment table, each entry of which contains the physical address of the page table for the segment. Page tables are stored in the main memory and consists of **2 byte page table entries**. What is the minimum page size in bytes so that the page table for a segment requires at most one page to store it?

Exercise question 3

Given-

- Virtual Address Space = Process size = 2^{16} bytes
- Physical Address Space = Main Memory size = 2^{16} bytes
- Process is divided into 8 equal size segments
- Page table entry size = 2 bytes

Let page size = n bytes.

Now, since page table has to be stored into a single page, so we must have-

$$\text{Size of page table} \leq \text{Page size}$$

Exercise question 3



Size of Each Segment-

Size of each segment

= Process size / Number of segments

= 2^{16} bytes / 8

= 2^{16} bytes / 2^3

= 2^{13} bytes

= 8 KB

Number of Pages Of Each Segment-

Number of pages each segment is divided

= Size of segment / Page size

= 8 KB / n bytes

= (8K / n) pages

Exercise question 3



Size of Each Page Table-

Size of each page table

= Number of entries in page table x
Page table entry size

= Number of pages the segment is
divided x 2 bytes

= $(8K / n) \times 2$ bytes

= $(16K / n)$ bytes

Page Size-

Substituting values in the above
condition, we get-

$(16K / n) \text{ bytes} \leq n \text{ bytes}$

$(16K / n) \leq n$

$n^2 \geq 16K$

$n^2 \geq 2^{14}$

$n \geq 2^7$

Thus, minimum page size possible = 2^7
bytes = 128 bytes.

Practice question



Considering previous problem, give the division of virtual address.